Ansible для PostgreSQL: Дополнительный материал

Дополнительный материал к курсу PostgreSQL Advanced

Dmitrii Zolotov

2026-02-18

1 Ansible для PostgreSQL: Руководство для DBA
1.1 Оглавление
1.2 Введение
1.2.1 О чём этот материал
1.2.2 Предварительные знания
1.3 Cloud-init: подготовка виртуальных машин
1.3.1 Почему это важно
1.3.2 Как это работает
1.3.3 Практика
1.3.4 Типичные ошибки и решения
1.3.5 Что важно запомнить
1.4 Ansible: архитектура и ключевые концепции
1.4.1 Почему это важно
1.4.2 Как это работает
1.4.3 Практика
1.4.4 Типичные ошибки и решения
1.4.5 Что важно запомнить
1.5 Модули Ansible
1.5.1 Почему это важно
1.5.2 Как это работает
1.5.3 Практика
1.5.4 Типичные ошибки и решения

1.5.5 Что важно запомнить

1.6 Playbooks и Roles

1.6.1 Почему это важно

1.6.2 Как это работает

1.6.3 Практика

1.6.4 Что важно запомнить

1.7 Ansible Galaxy и экосистема

1.7.1 Почему это важно

1.7.2 Как это работает

1.7.3 Практика

1.7.4 Что важно запомнить

1.8 Ansible Tower и AWX

1.8.1 Почему это важно

1.8.2 Как это работает

1.8.3 Практика

1.8.4 Что важно запомнить

1.9 Terraform + Ansible: интеграция

1.9.1 Почему это важно

1.9.2 Как это работает

1.9.3 Практика

1.9.4 Сравнение подходов интеграции

1.9.5 Типичные ошибки и решения

1.9.6 Что важно запомнить

1.10 community.postgresql: управление PostgreSQL как кодом

1.10.1 Почему это важно

1.10.2 Как это работает

1.10.3 Практика

1.10.4 Типичные ошибки и решения

1.10.5 Что важно запомнить

1.11 Типичные ошибки и best practices

1.11.1 Почему это важно

1.11.2 Практика

1.11.3 Типичные ошибки и решения

1.11.4 Что важно запомнить

1.12 Практические рецепты и troubleshooting

1.12.1 Шпаргалка команд

1.12.2 Карта принятия решений

1.12.3 Диагностика общих проблем

1.12.4 Полный пайплайн: от ВМ до PostgreSQL

1.13 Заключение

1.13.1 Что вы теперь знаете

1.13.2 Что дальше

1.14 Полезные ресурсы

1.14.1 Официальная документация

1.14.2 Инструменты

1.14.3 Книги и статьи

1 Ansible для PostgreSQL: Руководство для DBA

Дополнительный материал к курсу PostgreSQL Advanced Автор: Dmitrii Zolotov

---

1.1 Оглавление

1. Введение
2. Cloud-init: подготовка виртуальных машин
3. Ansible: архитектура и ключевые концепции
4. Модули Ansible
5. Playbooks и Roles
6. Ansible Galaxy и экосистема
7. Ansible Tower и AWX
8. Terraform + Ansible: интеграция
9. community.postgresql: управление PostgreSQL как кодом
10. Типичные ошибки и best practices
11. Практические рецепты и troubleshooting
12. Заключение
13. Полезные ресурсы

1.2 Введение

1.2.1 О чём этот материал

Этот документ — самостоятельное руководство по автоматизации управления PostgreSQL с помощью Ansible. Он не повторяет слайды презентации, а **углубляет каждую тему**: объясняет внутренние механизмы, даёт production-ready примеры, разбирает типичные ошибки и предлагает таблицы решений для выбора подходов.

Вы узнаете, как пройти полный путь от создания виртуальной машины до работающего PostgreSQL-кластера с автоматическими бэкапами, репликацией и мониторингом — и всё это описано как код, версионировано в Git и воспроизводимо на любом количестве серверов.

1.2.2 Предварительные знания

Предполагается, что вы знакомы с:

- Администрированием PostgreSQL (установка, конфигурация, pg_hba.conf, репликация)

- Базовой работой с Linux (SSH, пакетный менеджер, systemd)
- Основами YAML-синтаксиса
- Концепцией Infrastructure as Code (желательно, но не обязательно)

1.3 Cloud-init: подготовка виртуальных машин

1.3.1 Почему это важно

Представьте: вы развернули через Terraform 10 виртуальных машин для PostgreSQL-кластера. Машины созданы, IP-адреса получены, диски подключены. Но внутри — голая операционная система. Нужно:

- создать пользователя `deploy` с SSH-ключом
- обновить пакеты
- установить Python (для Ansible)
- подключить репозиторий PostgreSQL PGDG
- настроить файрвол

Делать это вручную через SSH на каждом сервере? Для одного — терпимо. Для десяти — уже боль. А через месяц нужно пересоздать стенд — и всё заново.

Типичный сценарий:

DBA получает запрос: «Нужен staging-кластер из 3 нод для тестирования миграции на PostgreSQL 17». Без автоматизации это 2-3 часа ручной работы. С cloud-init + Terraform — `terraform apply`, и через 5 минут серверы готовы к приёму Ansible.

Что изменится:

Было	Стало
SSH на каждый сервер, ручная настройка	Единый YAML-файл, автоматический bootstrap
Каждый сервер «уникальная снежинка»	Все серверы идентичны
Настройка 10 серверов: 2-3 часа	Настройка 10 серверов: 5 минут
Документация устаревает	Код **является** документацией

1.3.2 Как это работает

Cloud-init — индустриальный стандарт инициализации виртуальных машин при первом запуске. Поддерживается всеми основными облаками: AWS, GCP, Azure, Yandex Cloud, а также локальными платформами (Proxmox, OpenStack).

Аналогия для DBA:

Cloud-init — это как `initdb` для операционной системы. Как `initdb` создаёт начальную структуру кластера PostgreSQL (каталоги, конфиги, pg_hba.conf), cloud-init создаёт начальную конфигурацию ОС (пользователи, пакеты, сеть). И так же, как `initdb` запускается **один раз** при создании кластера, cloud-init работает **только при первом запуске** ВМ.

Внутренняя механика — 5 стадий:

Cloud-init работает в строго определённом порядке при загрузке системы:

1. **Detect (ds-identify)** — определяет облачную платформу. Cloud-init опрашивает metadata-сервис (обычно `http://169.254.169.254`) и понимает, что это Yandex Cloud, AWS или другой провайдер
2. **Local** — настраивает сеть до получения полного доступа к metadata-сервису. На этом этапе используется DHCP-fallback
3. **Network** — получает полные user-data с metadata-сервиса. Обрабатывает SSH-ключи, создаёт пользователей, монтирует диски
4. **Config** — выполняет runcmd-команды, настраивает timezone, NTP
5. **Final** — устанавливает пакеты, запускает скрипты пользователя, при наличии модуля `cc_ansible` — устанавливает и запускает Ansible

Когда это НЕ подходит:

- Текущее управление конфигурацией (для этого — Ansible)
- Развёртывание приложений после первого запуска
- Оркестрация кластеров (cloud-init не знает о других нодах)
- Любые изменения после первого запуска ВМ

1.3.3 Практика

1.3.3.1 Базовый сценарий: подготовка ВМ для PostgreSQL

```
#cloud-config
# ВАЖНО: файл ОБЯЗАТЕЛЬНО начинается с #cloud-config

# Создаём пользователя deploy для Ansible
users:
  - name: deploy
    sudo: ALL=(ALL) NOPASSWD:ALL    # Ansible нужен sudo без пароля
    shell: /bin/bash
    ssh_authorized_keys:
      - ssh-rsa AAAAB3NzaC1yc2E...  # Ваш публичный SSH-ключ

# Обновляем индекс пакетов
package_update: true

# Устанавливаем минимальный набор для работы Ansible
packages:
  - python3                          # Ansible требует Python на managed nodes
  - python3-pip                      # Для установки psycopg2 позже
  - python3-psycopg2                 # Для модулей community.postgresql
  - sudo                             # На случай если не установлен
  - curl                             # Утилиты
  - gnupg                            # Для работы с GPG-ключами репозиториев
```

Применить:

В Yandex Cloud cloud-init передаётся через metadata:

```
resource "yandex_compute_instance" "db" {
  name = "pg-primary"

  metadata = {
    ssh-keys  = "ubuntu:${file("~/.ssh/id_rsa.pub")}"
    user-data = file("cloud-init.yaml")
  }
  # ... остальная конфигурация
}
```

Проверить результат:

```
# После создания ВМ подключаемся и проверяем
ssh deploy@<IP>

# Проверяем, что cloud-init завершился успешно
cloud-init status
# Ожидаемый вывод:
# status: done

# Проверяем логи (если нужна диагностика)
sudo cat /var/log/cloud-init-output.log
```

1.3.3.2 Продвинутый сценарий: полный bootstrap PostgreSQL-сервера

```yaml
#cloud-config
package_update: true

# Подключаем официальный репозиторий PostgreSQL PGDG
apt:
  sources:
    pgdg:
      source: "deb http://apt.postgresql.org/pub/repos/apt $RELEASE-pgdg main"
      keyid: "ACCC4CF8"              # GPG-ключ репозитория PGDG
      keyserver: "keyserver.ubuntu.com"

# Создаём пользователя для Ansible
users:
  - name: deploy
    sudo: ALL=(ALL) NOPASSWD:ALL
    shell: /bin/bash
    ssh_authorized_keys:
      - ssh-rsa AAAAB3NzaC1yc2E...

# Устанавливаем всё необходимое
packages:
  - postgresql-17                    # Сам PostgreSQL
  - postgresql-client-17             # Клиентские утилиты
  - python3-psycopg2                 # Для Ansible-модулей
  - python3                          # Python для Ansible

# Настраиваем отдельный диск для данных PostgreSQL
disk_setup:
  /dev/vdb:                          # Второй диск ВМ
    table_type: gpt
    layout: true                     # Одна партиция на весь диск
    overwrite: false                 # Не трогать если уже размечен

fs_setup:
  - label: pgdata
    filesystem: ext4
    device: /dev/vdb1
    overwrite: false

mounts:
  - ["/dev/vdb1", "/var/lib/postgresql", "ext4", "defaults,nofail", "0", "2"]

# Команды выполняются после установки пакетов
runcmd:
  - chown -R postgres:postgres /var/lib/postgresql
  - systemctl enable postgresql
  - systemctl start postgresql
```

1.3.3.3 Сценарий с ansible-pull

Начиная с cloud-init **22.3** (текущая стабильная версия: **25.3**, сентябрь 2025), доступен встроенный модуль `cc_ansible`. Модуль запускается с частотой `PER_INSTANCE` — то есть **только при первом запуске** ВМ.

```
#cloud-config
ansible:
  install_method: pip                 # pip или distro (из системных пакетов)
  package_name: ansible-core         # Какой пакет устанавливать
  run_user: ubuntu                   # Под каким пользователем запускать
        (только с install_method: pip!)
  pull:
    url: "https://github.com/org/pg-ansible.git"
    playbook_name: postgresql.yml     # Один плейбук (НЕЛЬЗЯ одновременно с
        playbook_names!)
    accept_host_key: true
    full: true                        # Полный clone (не shallow)
    tags: [postgresql, base_setup]   # Запустить только указанные теги
```

Что произойдёт:

1. Cloud-init установит `ansible-core` через pip
2. Клонирует указанный Git-репозиторий
3. Запустит `ansible-pull` с указанным плейбуком
4. Плейбук выполнится **локально** на этой же машине

Важно: `ansible-pull` не требует отдельного control node — сервер сам себя конфигурирует. Это удобно для автомасштабирования, но сложнее отлаживать.

Ограничения cc_ansible (знать обязательно):

- `run_user` работает **только с** `install_method: pip`. При `install_method: distro` команды запускаются от root
- `playbook_name` и `playbook_names` — **взаимоисключающие** параметры. Одновременное использование вызывает ошибку валидации
- Несколько плейбуков в одном вызове требуют Ansible >= 2.12 и параметр `playbook_names` (множественное число)
- Galaxy-коллекции, которые устанавливает сам плейбук, могут быть не найдены (известный баг). Предустанавливайте их через секцию `galaxy.actions:` в cloud-config

Yandex Cloud — особенности datasource:

Yandex Cloud не имеет нативного datasource и использует EC2-совместимый API. Требуется явная конфигурация в образе ВМ:

```yaml
# /etc/cloud/cloud.cfg.d/99-yandex.cfg на базовом образе
datasource_list: [Ec2, None]
datasource:
  Ec2:
    strict_id: false   # ОБЯЗАТЕЛЬНО для Yandex Cloud — EC2 API не полностью
        совместим
    metadata_urls:
      - http://169.254.169.254
```

Для дисков в Yandex Cloud: дополнительные диски именуются `/dev/vdb`, `/dev/vdc` (не `/dev/sdb`), а стабильные пути — через `/dev/disk/by-id/` с префиксом `virtio-<disk-name>`.

1.3.4 Типичные ошибки и решения

Симптом	Вероятная причина	Диагностика	Решение
ВМ создана, но пакеты не установлены	Файл не начинается с `#cloud-config`	`cat /var/log/cloud-init.log`	Добавить `#cloud-config` в первую строку
`cloud-init status: error`	Ошибка синтаксиса YAML	`cloud-init schema --config-file cloud-init.yaml`	Проверить YAML-валидатором
Повторный запуск не применяет изменения	Cloud-init работает однократно	`ls /var/lib/cloud/instance/sem/`	Пересоздать ВМ или `sudo cloud-init clean && sudo cloud-init init`
Диск не примонтирован	Неверное имя устройства	`lsblk` на работающей ВМ	Уточнить имя диска для вашего облака (`/dev/vdb` vs `/dev/sdb`)
SSH-ключ не работает	Ключ в неверном формате	`cat /home/deploy/.ssh/authorized_keys`	Проверить формат: `ssh-rsa AAAA...` или `ssh-ed25519 AAAA...`

1.3.5 Что важно запомнить

- Cloud-init работает **один раз** при создании ВМ — это Day 0 инструмент
- Формат файла обязательно начинается с `#cloud-config`
- Модуль `cc_ansible` (cloud-init 22.3+) позволяет запускать Ansible-pull автоматически
- Для повторной настройки используйте Ansible, не cloud-init
- Всегда проверяйте `cloud-init status` после создания ВМ
- Для PostgreSQL: выносите `$PGDATA` на отдельный диск через `disk_setup` + `mounts`

Используйте, когда:

- Нужно подготовить ВМ для Ansible (Python, SSH, пользователи)
- Однократная начальная настройка ОС
- Bootstrap PostgreSQL в новом окружении

Не используйте, когда:

- Нужно управлять конфигурацией после первого запуска (используйте Ansible)
- Нужна оркестрация между несколькими серверами
- Нужно обновлять конфигурацию PostgreSQL

1.4 Ansible: архитектура и ключевые концепции

1.4.1 Почему это важно

Вы — DBA. У вас 1 сервер PostgreSQL, и вы прекрасно справляетесь через SSH. Потом серверов стало 3 (primary + 2 replicas). Потом 10 (3 кластера по разным проектам). Потом 30 (staging, dev, prod для каждого).

Проблема «снежинок» (snowflake servers):

Каждый сервер настраивался в разное время, разными людьми. На одном `shared_buffers = 2GB`, на другом — `2048MB`. На одном стоит `pg_stat_statements`, на другом — забыли. На одном в pg_hba.conf есть правило для мониторинга, на другом — нет. Конфигурация **дрейфует**: серверы, которые должны быть одинаковыми, постепенно расходятся.

Факты индустрии:

- По данным Puppet State of DevOps Report, организации, использующие Configuration Management, тратят на **60% меньше времени** на незапланированную работу
- Среднее время восстановления после сбоя сокращается в **24 раза** при использовании IaC

- 72% простоев вызваны конфигурационными ошибками (Gartner)

Что изменится:

Было	Стало
SSH на каждый сервер	Одна команда: `ansible-playbook site.yml`
Документация в Wiki (устарела)	Код = документация (всегда актуальна)
«Кто менял конфиг?» — неизвестно	`git log` покажет кто, когда и зачем
Откат — ручной	`git revert` + `ansible-playbook site.yml`

1.4.2 Как это работает

Ansible — open-source платформа автоматизации от Red Hat. Создана Michael DeHaan в 2012 году. Текущая версия: ansible-core 2.18+ (ansible-core 2.19 привнёс Data Tagging — крупнейшее обновление шаблонизатора со времён внедрения коллекций).

Аналогия для DBA:

Ansible для инфраструктуры — то же, что `ALTER SYSTEM` для PostgreSQL. Вы описываете **желаемое состояние** (`shared_buffers = '4GB'`), а система сама определяет, нужно ли что-то менять. Если параметр уже установлен — ничего не произойдёт (идемпотентность). Если нет — Ansible применит изменение.

Архитектура:

```
┌──────────────────────────────────────────────────────────────────────┐
│                        CONTROL NODE                                  │
│              (ваш ноутбук или CI/CD сервер)                          │
│                                                                      │
│   ┌──────────┐   ┌──────────┐   ┌──────────┐   ┌──────────────┐    │
│   │ Inventory │   │ Playbook │   │  Roles   │   │ansible.cfg │    │
│   │ hosts.yml │   │ site.yml │   │postgresql│   │            │    │
│   └──────┬────┘   └────┬─────┘   └────┬─────┘   └──────────────┘    │
│          │             │              │                              │
│          └─────────────┴──────────────┘                              │
│                        │                                             │
│                ┌───────▼────────┐                                    │
│                │  Ansible       │                                    │
│                │  Engine        │                                    │
│                └────────┬───────┘                                    │
└────────────────────────┼────────────────────────────────────────────┘
                         │ SSH (порт 22)
             ┌───────────┼────────────┐
             │           │            │
      ┌──────▼──────┐ ┌──▼──────┐ ┌──▼──────┐
      │ db-primary│ │db-repl1 │ │db-repl2 │
      │ PG Primary│ │PG Replica│ │PG Replica│
      │ 10.0.1.10 │ │10.0.1.11│ │10.0.1.12│
      └─────────────┘ └──────────┘ └──────────┘
         MANAGED NODES (только SSH + Python)
```

Ключевой принцип — Agentless:

В отличие от Puppet или Chef, Ansible **не устанавливает агенты** на управляемые серверы. Всё что нужно — SSH-доступ и Python 3. Для DBA это критически важно: ни один лишний процесс не забирает ресурсы у PostgreSQL.

Внутренняя механика выполнения задачи:

1. Ansible читает inventory — узнаёт, какие хосты в группе `postgresql`
2. Подключается по SSH к каждому хосту параллельно (по умолчанию — 5 одновременных соединений, настраивается через `forks`)
3. Копирует модуль (Python-скрипт) на удалённый хост в `/tmp`
4. Выполняет модуль, который проверяет текущее состояние и приводит к желаемому
5. Модуль возвращает JSON-результат: `changed: true/false`
6. Ansible удаляет временные файлы
7. Переходит к следующей задаче

1.4.3 Практика

1.4.3.1 Инвентарь: INI vs YAML

INI-формат (простой, подходит для небольших инфраструктур):

```
# inventory/hosts.ini
[pg_primary]
db-primary ansible_host=10.0.1.10

[pg_replicas]
db-replica1 ansible_host=10.0.1.11
db-replica2 ansible_host=10.0.1.12

# Группа-родитель, объединяющая primary и replicas
[postgresql:children]
pg_primary
pg_replicas

# Общие переменные для всех хостов PostgreSQL
[postgresql:vars]
ansible_user=deploy
ansible_become=true
postgresql_version=17
```

YAML-формат (более гибкий, рекомендуется для сложных инфраструктур):

```yaml
# inventory/hosts.yml
all:
  children:
    postgresql:
      children:
        pg_primary:
          hosts:
            db-primary:
              ansible_host: 10.0.1.10
              postgresql_role: primary    # Кастомная переменная
        pg_replicas:
          hosts:
            db-replica1:
              ansible_host: 10.0.1.11
              postgresql_role: replica
            db-replica2:
              ansible_host: 10.0.1.12
              postgresql_role: replica
      vars:
        ansible_user: deploy
        ansible_become: true
        postgresql_version: "17"
```

Проверить инвентарь:

```bash
# Список всех хостов
ansible-inventory -i inventory/hosts.yml --list
# Ожидаемый вывод: JSON с группами и переменными

# Граф инвентаря
ansible-inventory -i inventory/hosts.yml --graph
# Ожидаемый вывод:
# @all:
#   |--@postgresql:
#   |  |--@pg_primary:
#   |  |  |--db-primary
#   |  |--@pg_replicas:
#   |  |  |--db-replica1
#   |  |  |--db-replica2
```

1.4.3.2 Конфигурация ansible.cfg

```
# ansible.cfg — в корне проекта
[defaults]
inventory = inventory/hosts.yml      # Путь к инвентарю по умолчанию
roles_path = roles                   # Где искать роли
collections_paths = collections      # Где искать коллекции
host_key_checking = false            # Отключить проверку SSH-ключей (для dev)
retry_files_enabled = false          # Не создавать .retry файлы
forks = 10                           # Параллельность: 10 хостов одновременно

[privilege_escalation]
become = true                        # Включить sudo по умолчанию
become_method = sudo
become_user = root

[ssh_connection]
pipelining = true                    # Ускорение: меньше SSH-операций на задачу
ssh_args = -o ControlMaster=auto -o ControlPersist=60s
```

Pipelining уменьшает количество SSH-операций: вместо копирования модуля в файл и его запуска, Ansible передаёт модуль через stdin. Ускорение: **1.5-2x** на большом количестве задач.

1.4.4 Типичные ошибки и решения

Симптом	Вероятная причина	Диагностика	Решение
`Permission denied (publickey)`	SSH-ключ не добавлен	`ssh -v deploy@<IP>`	Проверить `~/.ssh/authorized_keys` на managed node
`Timeout` при подключении	Файрвол блокирует SSH	`nc -zv <IP> 22`	Открыть порт 22 в security group / файрволе
`Missing sudo password`	Пользователь требует пароль для sudo	`sudo -l` на managed node	Настроить `NOPASSWD` в sudoers или использовать `--ask-become-pass`
`Python interpreter not found`	Python не установлен	`ssh deploy@<IP> python3 --version`	Установить Python через cloud-init или вручную

1.4.5 Что важно запомнить

- Ansible — **agentless**: только SSH + Python на managed nodes
- **Идемпотентность**: повторный запуск не вносит изменений, если состояние уже соответствует желаемому
- **Декларативный подход**: описываете «что», а не «как»
- `ansible.cfg` ищется в текущей директории → в `~/.ansible.cfg` → в `/etc/ansible/ansible.cfg`
- `pipelining = true` — включайте всегда для ускорения
- `forks` по умолчанию = 5, увеличивайте для больших инфраструктур (50-100 для 500+ хостов)

1.5 Модули Ansible

1.5.1 Почему это важно

Модуль — это «кирпичик» автоматизации в Ansible. Каждый модуль выполняет одну задачу: установить пакет, создать файл, перезапустить сервис. Ansible Core включает **70+ встроенных модулей**, а в Ansible Galaxy доступно более **10 000** модулей от сообщества.

Аналогия для DBA:

Модули Ansible — это как SQL-команды в PostgreSQL. `apt` — это `CREATE DATABASE`, `service` — это `ALTER SYSTEM`, `template` — это `COPY ... FROM`. Каждый модуль идемпотентен: `CREATE DATABASE IF NOT EXISTS` не создаст дубликат, и `ansible.builtin.apt` не переустановит уже установленный пакет.

1.5.2 Как это работает

Внутренняя механика модуля:

1. Ansible сериализует параметры задачи в JSON
2. Передаёт JSON и Python-код модуля на managed node через SSH
3. Модуль выполняется на managed node:
   - Проверяет текущее состояние (`apt list --installed | grep postgresql-17`)
   - Сравнивает с желаемым (`state: present`)
   - Если нужно — вносит изменения (`apt install postgresql-17`)
4. Возвращает JSON с результатом: `{"changed": true/false, "msg": "..."}`

Почему это важно:

```
# Первый запуск — пакет устанавливается
$ ansible db-primary -m ansible.builtin.apt -a "name=postgresql-17
        state=present" --become
db-primary | CHANGED => {"changed": true}

# Второй запуск — пакет уже есть, ничего не происходит
$ ansible db-primary -m ansible.builtin.apt -a "name=postgresql-17
        state=present" --become
db-primary | SUCCESS => {"changed": false}
```

1.5.3 Практика

1.5.3.1 Категории модулей для DBA

Управление пакетами:

```yaml
# Установка PostgreSQL из репозитория PGDG
- name: Установить PostgreSQL 17
  ansible.builtin.apt:
    name:
      - postgresql-17
      - postgresql-client-17
      - python3-psycopg2         # Требуется для community.postgresql
    state: present
    update_cache: true           # apt update перед установкой
    cache_valid_time: 3600       # Кешировать apt update на 1 час
```

Управление сервисами:

```yaml
# Запуск и включение PostgreSQL
- name: PostgreSQL запущен и включён в автозагрузку
  ansible.builtin.service:
    name: postgresql
    state: started               # started/stopped/restarted/reloaded
    enabled: true                # Включить в systemd
```

Работа с файлами (template):

```yaml
# Генерация postgresql.conf из Jinja2-шаблона
- name: Настроить postgresql.conf
  ansible.builtin.template:
    src: postgresql.conf.j2      # Шаблон из каталога templates/
    dest: /etc/postgresql/17/main/postgresql.conf
    owner: postgres
    group: postgres
    mode: "0644"
    backup: true                 # Создать .bak перед перезаписью
  notify: Restart PostgreSQL     # Вызвать handler при изменении
```

Редактирование строк в файлах:

```yaml
# Добавить строку в pg_hba.conf (если её ещё нет)
- name: Разрешить мониторинг
  ansible.builtin.lineinfile:
    path: /etc/postgresql/17/main/pg_hba.conf
    line: "host all monitoring 10.0.0.0/8 scram-sha-256"
    state: present
  notify: Reload PostgreSQL
```

Выполнение команд:

```yaml
# Проверить версию PostgreSQL (shell нужен для пайпов)
- name: Проверить версию PostgreSQL
  ansible.builtin.command:
    cmd: pg_lsclusters --no-header
  register: pg_clusters          # Сохранить вывод в переменную
  changed_when: false            # Эта команда ничего не меняет

- name: Показать результат
  ansible.builtin.debug:
    msg: "{{ pg_clusters.stdout }}"
```

1.5.3.2 FQCN: полные имена модулей

С Ansible 2.10+ модули объединены в **коллекции** и используют Fully Qualified Collection Name (FQCN):

Старый синтаксис	FQCN	Коллекция
`apt`	`ansible.builtin.apt`	ansible-core
`service`	`ansible.builtin.service`	ansible-core
`template`	`ansible.builtin.template`	ansible-core
`postgresql_db`	`community.postgresql.postgresql_db`	community.postgresql

Всегда используйте FQCN. Это не просто конвенция — `ansible-lint` помечает короткие имена как deprecated. В ansible-core 2.19+ короткие имена модулей могут вызвать предупреждения.

1.5.4 Типичные ошибки и решения

Симптом	Вероятная причина	Диагностика	Решение
`changed` всегда `true`	Используете `shell` / `command` вместо модуля	Проверить логику задачи	Заменить на специализированный модуль (apt, service, etc.)
`No module named...`	Коллекция не установлена	`ansible-galaxy collection list`	`ansible-galaxy collection install community.postgresql`
`Permission denied` при работе с файлами	Нет become или неверный owner	`ls -la /path/to/file`	Добавить `become: true` или указать правильного `owner`

1.5.5 Что важно запомнить

- Модули — идемпотентные единицы работы Ansible
- **Всегда используйте FQCN**: `ansible.builtin.apt`, не `apt`
- `shell` и `command` — **не идемпотентны**, используйте только когда нет специализированного модуля
- Если используете `command` / `shell`, добавляйте `changed_when` и `creates` / `removes` для идемпотентности
- `notify` + handlers — правильный способ перезапуска сервисов

1.6 Playbooks и Roles

1.6.1 Почему это важно

Модуль — кирпичик. Playbook — чертёж. Role — готовый блок стены. Без структурирования автоматизации код превращается в длинный неподдерживаемый скрипт. С ролями вы получаете переиспользуемые компоненты, которые легко тестировать, версионировать и передавать коллегам.

Типичный сценарий:

Вы написали плейбук на 500 строк для установки PostgreSQL. Через месяц нужен такой же для другого проекта, но с PostgreSQL 16 вместо 17. Без ролей — копипаста. С ролями — параметризация через переменные.

1.6.2 Как это работает

Playbook — YAML-файл с упорядоченным списком plays (наборов задач), выполняемых на группах хостов.

Аналогия для DBA:

Playbook — это как SQL-скрипт миграции: набор команд, выполняемых последовательно. Как `BEGIN; CREATE TABLE ...; INSERT ...; COMMIT;`, playbook описывает шаги от начального состояния к желаемому.

Handlers — специальные задачи, вызываемые через `notify`. Они выполняются **один раз в конце play**, даже если несколько задач вызвали один и тот же handler.

Аналогия: Handler — это как `COMMIT` после серии изменений. Вы меняете `postgresql.conf` и `pg_hba.conf` (два `notify: Restart PostgreSQL`), но рестарт произойдёт **один раз**, а не два.

1.6.3 Практика

1.6.3.1 Структура production-проекта

```
ansible-postgresql/
├── ansible.cfg                     # Конфигурация Ansible
├── inventory/
|   ├── hosts.yml                   # Инвентарь
|   ├── group_vars/
|   |   ├── all.yml                 # Переменные для всех хостов
|   |   └── postgresql.yml          # Переменные для группы postgresql
|   └── host_vars/
|       └── db-primary.yml          # Переменные для конкретного хоста
├── playbooks/
|   └── site.yml                    # Главный плейбук
├── roles/
|   └── postgresql/                 # Кастомная роль
|       ├── tasks/main.yml          # Задачи
|       ├── handlers/main.yml       # Handlers (restart, reload)
|       ├── templates/              # Jinja2-шаблоны
|       |   ├── postgresql.conf.j2
|       |   └── pg_hba.conf.j2
|       ├── defaults/main.yml       # Значения переменных по умолчанию
|       └── meta/main.yml           # Зависимости роли
└── requirements.yml                # Зависимости из Galaxy
```

1.6.3.2 Плейбук с handlers

```yaml
---
- name: Настройка PostgreSQL кластера
  hosts: postgresql
  become: true

  vars:
    pg_version: "17"
    pg_max_connections: 200
    pg_shared_buffers: "2GB"

  tasks:
    - name: Установить PostgreSQL
      ansible.builtin.apt:
        name:
          - "postgresql-{{ pg_version }}"
          - python3-psycopg2
        state: present
        update_cache: true

    - name: Настроить postgresql.conf
      ansible.builtin.template:
        src: postgresql.conf.j2
        dest: "/etc/postgresql/{{ pg_version }}/main/postgresql.conf"
        owner: postgres
        group: postgres
      notify: Restart PostgreSQL           # Вызовет handler

    - name: Настроить pg_hba.conf
      ansible.builtin.template:
        src: pg_hba.conf.j2
        dest: "/etc/postgresql/{{ pg_version }}/main/pg_hba.conf"
        owner: postgres
        group: postgres
      notify: Reload PostgreSQL            # Другой handler

  handlers:
    - name: Restart PostgreSQL
      ansible.builtin.service:
        name: postgresql
        state: restarted

    - name: Reload PostgreSQL
      ansible.builtin.service:
        name: postgresql
        state: reloaded
```

1.6.3.3 Шаблон Jinja2: postgresql.conf.j2

```
# Ansible managed - do not edit manually
# Generated: {{ ansible_date_time.iso8601 }}

listen_addresses = '{{ pg_listen_addresses | default("localhost") }}'
port = {{ pg_port | default(5432) }}
max_connections = {{ pg_max_connections | default(100) }}

# Память — автоматический расчёт от RAM хоста
shared_buffers = {{ pg_shared_buffers | default((ansible_memtotal_mb * 0.25) |
int ~ "MB") }}
effective_cache_size = {{ pg_effective_cache_size |
default((ansible_memtotal_mb * 0.75) | int ~ "MB") }}
work_mem = {{ pg_work_mem | default("4MB") }}

# WAL
wal_level = {{ pg_wal_level | default("replica") }}
max_wal_senders = {{ pg_max_wal_senders | default(10) }}
max_replication_slots = {{ pg_max_replication_slots | default(10) }}

# Логирование
log_min_duration_statement = {{ pg_log_min_duration | default(1000) }}
log_checkpoints = on
log_connections = on
log_disconnections = on
```

1.6.3.4 Роль: структура и использование

`roles/postgresql/tasks/main.yml`:

```yaml
---
- name: Добавить репозиторий PGDG
  ansible.builtin.apt_repository:
    repo: "deb http://apt.postgresql.org/pub/repos/apt
        {{ ansible_distribution_release }}-pgdg main"
    state: present

- name: Установить PostgreSQL
  ansible.builtin.apt:
    name:
      - "postgresql-{{ postgresql_version }}"
      - "postgresql-client-{{ postgresql_version }}"
      - python3-psycopg2
    state: present
    update_cache: true

- name: Развернуть конфигурацию
  ansible.builtin.template:
    src: "{{ item.src }}"
    dest: "{{ item.dest }}"
    owner: postgres
    group: postgres
    mode: "0640"
  loop:
    - { src: postgresql.conf.j2, dest: "/etc/postgresql/
        {{ postgresql_version }}/main/postgresql.conf" }
    - { src: pg_hba.conf.j2, dest: "/etc/postgresql/{{ postgresql_version }}/
        main/pg_hba.conf" }
  notify: Restart PostgreSQL
```

`roles/postgresql/defaults/main.yml`:

```yaml
---
# Значения по умолчанию — переопределяются в group_vars
postgresql_version: "17"
postgresql_listen_addresses: "localhost"
postgresql_max_connections: 100
postgresql_shared_buffers: "128MB"
```

Использование роли в плейбуке:

```yaml
---
- name: Развернуть PostgreSQL
  hosts: postgresql
  become: true
  roles:
    - role: postgresql
      vars:
        postgresql_version: "17"
        postgresql_max_connections: 200
        postgresql_shared_buffers: "2GB"
```

1.6.3.5 Приоритет переменных

Ansible имеет **22 уровня приоритета переменных**. Для DBA важны основные:

Приоритет	Источник	Пример
Высший	`-e` (extra vars в CLI)	`ansible-playbook site.yml -e pg_version=16`
↑	Переменные в task	`vars:` внутри задачи
↑	Переменные в play	`vars:` в play
↑	`host_vars/`	`host_vars/db-primary.yml`
↑	`group_vars/`	`group_vars/postgresql.yml`
Низший	`defaults/main.yml` роли	`roles/postgresql/defaults/main.yml`

Правило: помещайте общие значения в `defaults/main.yml` роли, специфичные для окружения — в `group_vars/`, а для конкретного хоста — в `host_vars/`.

1.6.4 Что важно запомнить

- Handlers выполняются **один раз в конце play**, не после каждого `notify`
- `defaults/main.yml` — самый низкий приоритет, легко переопределить
- `-e` (extra vars) — самый высокий приоритет, перебивает всё
- Структурируйте проект: отдельные каталоги для inventory, roles, playbooks
- `serial: 1` — выполнять play по одному хосту за раз (для rolling updates)

1.7 Ansible Galaxy и экосистема

1.7.1 Почему это важно

Не изобретайте велосипед. Для PostgreSQL уже написаны десятки проверенных ролей и коллекций, которые покрывают 80% типичных задач DBA. Ansible Galaxy — это «pip для Ansible», публичный репозиторий с 36 000+ ролями и 3 900+ коллекциями.

1.7.2 Как это работает

Ansible Galaxy — хаб для обмена автоматизацией. Две сущности:

- **Роли** — переиспользуемые наборы задач/шаблонов для конкретной задачи
- **Коллекции** — пакеты из модулей, ролей, плагинов и документации (формат v2)

Аналогия для DBA:

Galaxy — это как pgxn.org (PostgreSQL Extension Network) или CPAN для Perl. Вместо написания расширения с нуля вы устанавливаете `pg_stat_statements`. Вместо написания роли — устанавливаете `geerlingguy.postgresql`.

1.7.3 Практика

1.7.3.1 requirements.yml: декларативные зависимости

```yaml
---
# requirements.yml — фиксируем зависимости проекта
roles:
  - name: geerlingguy.postgresql
    version: "4.0.3"                 # Всегда фиксируйте версию!

collections:
  - name: community.postgresql
    version: ">=4.2.0,<5.0.0"       # Семантическое версионирование
  - name: cloud.terraform
    version: ">=4.0.0"
  - name: community.general
    version: ">=9.0.0"
```

Установка:

```
ansible-galaxy install -r requirements.yml
# Ожидаемый вывод:
# Starting galaxy role install process
# - downloading role 'postgresql' from https://galaxy.ansible.com/...
# Starting galaxy collection install process
# Installing 'community.postgresql:4.2.0' to '/home/user/.ansible/collections'
```

1.7.3.2 Топ-роли для PostgreSQL

Роль	Назначение	Когда использовать
`geerlingguy.postgresql`	Установка и базовая настройка	Простые standalone-инстансы
`vitabaks.autobase`	HA кластер: Patroni + etcd + HAProxy	Production HA-кластеры
Собственная роль	Максимальная гибкость	Нестандартные конфигурации

Рекомендация: Начните с `geerlingguy.postgresql` для изучения, переходите на собственные роли с модулями `community.postgresql` для production.

1.7.4 Что важно запомнить

- **Всегда фиксируйте версии** в `requirements.yml`
- Коммитьте `requirements.yml` в Git, **не коммитьте** скачанные роли/коллекции
- Добавьте в `.gitignore`: `roles/geerlingguy.*`, `collections/`
- `community.postgresql` v4.2.0 — текущая стабильная версия коллекции для PostgreSQL

1.8 Ansible Tower и AWX

1.8.1 Почему это важно

Ansible CLI отлично работает для одного DBA с ноутбуком. Но когда команда растёт до 5+ человек, появляются вопросы: «Кто запустил плейбук?», «Какой версии плейбук использовался?», «Как ограничить доступ младших DBA к production?»

1.8.2 Как это работает

Ansible Tower — enterprise web UI и API для Ansible (платный, от Red Hat). **AWX** — его open-source upstream.

Ключевые возможности:

- **Web UI** — запуск плейбуков из браузера
- **RBAC** — ролевая модель доступа (кто может запускать что и где)
- **Credentials Vault** — безопасное хранение SSH-ключей и паролей
- **Job Scheduling** — cron-like планировщик (ежедневные backup, патчинг)
- **REST API** — интеграция с CI/CD (Jenkins, GitLab CI, GitHub Actions)
- **Audit & Logging** — полная история: кто, когда и что запустил

Аналогия для DBA: AWX для Ansible — как pgAdmin для PostgreSQL, но с RBAC и аудитом.

1.8.3 Практика

1.8.3.1 Таблица решений: Tower vs AWX vs CLI

Критерий	Ansible CLI	AWX	Ansible Tower
Стоимость	Бесплатно	Бесплатно	От $5000/год
Команда	1-2 DBA	3-10 DBA	10+ DBA
RBAC	Нет	Да	Да
Audit	`git log`	Встроенный	Встроенный + compliance
Поддержка	Community	Community	Red Hat Enterprise
Scheduling	cron	Встроенный	Встроенный
Рекомендация	Dev, обучение	Малый/средний бизнес	Enterprise

1.8.4 Что важно запомнить

- Начните с Ansible CLI, переходите на AWX/Tower при росте команды
- AWX — бесплатный, но bleeding edge (обновления каждые 2-3 недели)
- Tower — стабильный, с enterprise-поддержкой от Red Hat
- REST API позволяет интегрировать Ansible в любой CI/CD пайплайн

1.9 Terraform + Ansible: интеграция

1.9.1 Почему это важно

Terraform создаёт инфраструктуру (ВМ, сети, диски), но **не управляет ОС и ПО** внутри ВМ. Ansible управляет конфигурацией, но **не создаёт инфраструктуру**. Вместе они покрывают весь жизненный цикл:

```
Terraform (Day 0) → Cloud-init (Day 0) → Ansible (Day 1+)
   Создание ВМ         Bootstrap ОС        Настройка PostgreSQL
```

Аналогия для DBA: Terraform — это `CREATE TABLE`, Ansible — это `ALTER TABLE` и ежедневное обслуживание. Terraform описывает **структуру**, Ansible — **содержимое**.

1.9.2 Как это работает

Есть три способа передать информацию из Terraform в Ansible:

1. templatefile() + local_file (рекомендуется для PostgreSQL кластеров):

Terraform генерирует статический файл инвентаря через шаблон. Для PostgreSQL кластеров с фиксированной топологией (primary + N реплик) — это самый практичный подход: понятный, версионируемый, не требует дополнительных зависимостей.

2. terraform-provider-ansible + cloud.terraform:

Terraform создаёт ресурсы `ansible_host` и `ansible_group`, которые коллекция `cloud.terraform` читает как динамический инвентарь. Лучший выбор при **динамической** инфраструктуре (auto-scaling группы, эфемерные узлы) — для PostgreSQL кластеров применяется реже.

3. local-exec с ansible-playbook:

Terraform напрямую вызывает Ansible после создания ресурсов. **Не рекомендуется** — жёсткая связность, длинный `terraform apply`, сложная обработка ошибок.

1.9.3 Практика

1.9.3.1 Способ 1: terraform-provider-ansible

Terraform-конфигурация:

```
terraform {
  required_providers {
    ansible = {
      source  = "ansible/ansible"
      version = "~> 1.3.0"
    }
  }
}

# Регистрируем хост в Ansible-инвентаре
resource "ansible_host" "db_primary" {
  name   = yandex_compute_instance.db.network_interface.0.nat_ip_address
  groups = ["postgresql", "pg_primary"]     # Группы Ansible

  variables = {
    ansible_user                 = "deploy"
    ansible_ssh_private_key_file = "~/.ssh/id_rsa"
    postgresql_version           = "17"
    postgresql_role              = "primary"
  }
}

resource "ansible_group" "postgresql" {
  name     = "postgresql"
  children = ["pg_primary", "pg_replicas"]
  variables = {
    ansible_become = "true"
    db_port        = "5432"
  }
}
```

Inventory для Ansible (динамический):

```yaml
# inventory.yml
plugin: cloud.terraform.terraform_provider
project_path: /path/to/terraform/project
```

Установка и использование:

```
# Установить коллекцию
ansible-galaxy collection install cloud.terraform

# Создать инфраструктуру
terraform apply

# Запустить Ansible с динамическим инвентарём
ansible-playbook -i inventory.yml site.yml
```

1.9.3.2 Способ 2: templatefile() для статического инвентаря

```
# В Terraform
resource "local_file" "inventory" {
  content = templatefile("inventory.tftpl", {
    primary_ip  =
yandex_compute_instance.primary.network_interface.0.nat_ip_address
    replica_ips =
yandex_compute_instance.replica[*].network_interface.0.nat_ip_address
  })
  filename = "${path.module}/inventory.ini"
}
```

Шаблон `inventory.tftpl`:

```
[pg_primary]
${primary_ip} ansible_user=deploy

[pg_replicas]
%{ for ip in replica_ips ~}
${ip} ansible_user=deploy
%{ endfor ~}

[postgresql:children]
pg_primary
pg_replicas
```

1.9.4 Сравнение подходов интеграции

Критерий	`templatefile()`	`terraform-provider-ansible`	`local-exec`
Связность Terraform/Ansible	Средняя (файл)	Слабая (state)	**Высокая**
Топология кластера	Статическая (рекомендуется для PG)	Динамическая	Любая
Дополнительные зависимости	Нет	`cloud.terraform` collection	Ansible на CI-машине
Отладка	Просто (читаем файл)	Сложнее (`ansible-inventory --list`)	Трудно
CI/CD интеграция	Удобно (файл — артефакт)	Нужен доступ к state backend	Встроено
Производительность `terraform apply`	Быстро (файл генерируется мгновенно)	Быстро	**Медленно** (блокирует apply)
Рекомендация	✅ PostgreSQL кластеры	Auto-scaling инфраструктура	❌ Не рекомендуется

1.9.5 Типичные ошибки и решения

Симптом	Вероятная причина	Диагностика	Решение
Ansible не видит хосты	`cloud.terraform` не установлен	`ansible-galaxy collection list`	`ansible-galaxy collection install cloud.terraform`
`terraform apply` зависает	`ansible_playbook` ресурс блокирует	`terraform state list`	Запускать Ansible отдельно, не через `ansible_playbook` ресурс
IP-адрес изменился	ВМ пересоздана	`terraform show`	Инвентарь автоматически обновится при использовании провайдера

1.9.6 Что важно запомнить

- Terraform = Day 0 (инфраструктура), Ansible = Day 1+ (конфигурация). Никогда не смешивайте ответственность
- `templatefile()` + `local_file` — рекомендуется для PostgreSQL кластеров: фиксированная топология, максимальная простота
- `terraform-provider-ansible` + `cloud.terraform` — для динамических окружений (auto-scaling, эфемерные ноды)
- Не используйте `ansible_playbook` ресурс или `local-exec` с длинными плейбуками — блокирует `terraform apply` на 30-45 минут
- Разделяйте пайплайны: изменения инфраструктуры (Terraform) и изменения конфигурации (Ansible) — отдельные CI/CD задачи с разной частотой запуска

1.10 community.postgresql: управление PostgreSQL как кодом

1.10.1 Почему это важно

Это центральная тема для DBA. Коллекция `community.postgresql` содержит **20+ модулей**, которые позволяют управлять PostgreSQL декларативно: базы данных, пользователи, привилегии, pg_hba.conf, расширения, репликация — всё через YAML.

Типичный сценарий:

Новый проект. Нужно: создать базу, 3 пользователя (app, readonly, monitoring), настроить права, установить расширения, обновить pg_hba.conf. Вручную — 30 минут SQL-команд и редактирования файлов. С Ansible — 50 строк YAML, которые можно повторить на любом количестве серверов.

Что изменится:

Было	Стало
`CREATE USER app_user ...` в psql	`community.postgresql.postgresql_user` в YAML
Ручное редактирование pg_hba.conf	`community.postgresql.postgresql_pg_hba`
`ALTER SYSTEM SET ...` в psql	`community.postgresql.postgresql_alter_system`
Документация: «Не забудь создать extension»	Код автоматически создаёт все расширения

1.10.2 Как это работает

Установка:

```
# Установить коллекцию
ansible-galaxy collection install community.postgresql

# Или через requirements.yml
collections:
  - name: community.postgresql
    version: ">=4.2.0,<5.0.0"
```

Требования на managed node:

- PostgreSQL установлен и запущен
- Python-адаптер: `psycopg2 >= 2.5.1` **или** `psycopg3 >= 3.1.8`

```
# Установить psycopg2 (рекомендуется через системный пакет)
apt install python3-psycopg2

# Или через pip (для psycopg3)
pip install psycopg[binary]>=3.1.8
```

Подключение к PostgreSQL:

Модули подключаются к PostgreSQL двумя способами:

1. **Unix socket (peer auth)** — по умолчанию, когда `login_host` не указан:

```
- community.postgresql.postgresql_db:
    name: mydb
  become_user: postgres              # Подключение через unix socket
```

1. **TCP (password auth)** — когда указан `login_host`:

```
- community.postgresql.postgresql_db:
    name: mydb
    login_host: 10.0.1.10            # TCP-соединение
    login_user: postgres
    login_password: "{{ vault_pg_password }}"
```

Важно: `login_host: localhost` — это **TCP**, не unix socket! Для unix socket `login_host` не указывайте.

1.10.3 Практика

1.10.3.1 Создание баз данных и пользователей

```yaml
---
- name: Настройка PostgreSQL для приложения
  hosts: pg_primary
  become: true

  vars:
    app_db: myapp_production
    app_user: myapp_user
    readonly_user: readonly_user
    monitoring_user: monitoring

  tasks:
    # Создание базы данных
    - name: Создать базу данных
      community.postgresql.postgresql_db:
        name: "{{ app_db }}"
        encoding: UTF-8
        lc_collate: en_US.UTF-8
        template: template0          # Чистый шаблон для правильной кодировки
        state: present
      become_user: postgres

    # Создание пользователя приложения
    - name: Создать пользователя приложения
      community.postgresql.postgresql_user:
        name: "{{ app_user }}"
        password: "{{ vault_app_password }}"   # Из Ansible Vault
        db: "{{ app_db }}"
        state: present
      become_user: postgres

    # Назначение привилегий
    - name: Выдать права на базу
      community.postgresql.postgresql_privs:
        db: "{{ app_db }}"
        role: "{{ app_user }}"
        type: database
        privs: ALL
        state: present
      become_user: postgres

    # Создание readonly-пользователя
    - name: Создать readonly пользователя
      community.postgresql.postgresql_user:
        name: "{{ readonly_user }}"
        password: "{{ vault_readonly_password }}"
        state: present
      become_user: postgres

    - name: Выдать CONNECT на базу
      community.postgresql.postgresql_privs:
```

```yaml
        db: "{{ app_db }}"
        role: "{{ readonly_user }}"
        type: database
        privs: CONNECT
        state: present
      become_user: postgres

    - name: Выдать SELECT на все таблицы в public
      community.postgresql.postgresql_privs:
        db: "{{ app_db }}"
        role: "{{ readonly_user }}"
        schema: public
        type: table
        objs: ALL_IN_SCHEMA
        privs: SELECT
        state: present
      become_user: postgres

    # Установка расширений
    - name: Установить расширения
      community.postgresql.postgresql_ext:
        name: "{{ item }}"
        db: "{{ app_db }}"
        state: present
      loop:
        - pg_stat_statements
        - pgcrypto
        - uuid-ossp
      become_user: postgres
```

1.10.3.2 Управление pg_hba.conf

```yaml
- name: Разрешить подключение приложения
  community.postgresql.postgresql_pg_hba:
    dest: /etc/postgresql/17/main/pg_hba.conf
    contype: host
    databases: "{{ app_db }}"
    users: "{{ app_user }}"
    address: 10.0.2.0/24            # Подсеть приложения
    method: scram-sha-256           # Современный метод аутентификации
    state: present
  notify: Reload PostgreSQL

- name: Разрешить репликацию
  community.postgresql.postgresql_pg_hba:
    dest: /etc/postgresql/17/main/pg_hba.conf
    contype: host
    databases: replication
    users: replicator
    address: 10.0.1.0/24            # Подсеть кластера PostgreSQL
    method: scram-sha-256
  notify: Reload PostgreSQL

- name: Разрешить мониторинг
  community.postgresql.postgresql_pg_hba:
    dest: /etc/postgresql/17/main/pg_hba.conf
    contype: host
    databases: all
    users: monitoring
    address: 10.0.0.0/8
    method: scram-sha-256
  notify: Reload PostgreSQL
```

1.10.3.3 postgresql_alter_system: настройка параметров (PostgreSQL 14+)

```yaml
# Автоматический расчёт параметров на основе RAM хоста
# ВАЖНО: параметр называется 'param', не 'name' (в отличие от deprecated
        postgresql_set)
- name: Настройка shared_buffers (25% RAM)
  community.postgresql.postgresql_alter_system:
    param: shared_buffers            # Ключевое отличие от postgresql_set:
        здесь 'param', не 'name'
    value: "{{ (ansible_memtotal_mb * 0.25) | int }}MB"
  become_user: postgres
  notify: Restart PostgreSQL         # shared_buffers требует рестарта

- name: Настройка effective_cache_size (75% RAM)
  community.postgresql.postgresql_alter_system:
    param: effective_cache_size
    value: "{{ (ansible_memtotal_mb * 0.75) | int }}MB"
  become_user: postgres
  notify: Reload PostgreSQL          # effective_cache_size – reload достаточно

- name: Настройка work_mem
  community.postgresql.postgresql_alter_system:
    param: work_mem
    value: "{{ ((ansible_memtotal_mb * 0.05) / 200) | int }}MB"
  become_user: postgres
  notify: Reload PostgreSQL

- name: Включить логирование медленных запросов
  community.postgresql.postgresql_alter_system:
    param: log_min_duration_statement
    value: "1000"                    # 1 секунда
  become_user: postgres
  notify: Reload PostgreSQL

- name: Настройка WAL
  community.postgresql.postgresql_alter_system:
    param: wal_level
    value: replica
  become_user: postgres
  notify: Restart PostgreSQL         # wal_level требует рестарта

# Сброс параметра к default
- name: Сбросить work_mem к default
  community.postgresql.postgresql_alter_system:
    param: work_mem
    value: _RESET                    # Специальное значение для сброса (не
        'default')
  become_user: postgres
  notify: Reload PostgreSQL
```

Важно: `postgresql_alter_system` заменяет deprecated модуль `postgresql_set` (будет удалён в v5.0.0). Ключевые отличия:

- `postgresql_set` использовал `name:` для имени параметра
- `postgresql_alter_system` использует `param:` — частая ловушка при миграции
- Для сброса используйте `value: _RESET` (не `value: default`)
- `postgresql_alter_system` возвращает `restart_required: true`, когда параметр требует рестарта — используйте это для условного уведомления handlers

1.10.3.4 Сбор информации и мониторинг

```yaml
- name: Получить информацию о сервере
  community.postgresql.postgresql_info:
    filter:
      - version
      - databases
      - settings
  become_user: postgres
  register: pg_info

- name: Показать версию
  ansible.builtin.debug:
    msg: "PostgreSQL {{ pg_info.version.full }}"

- name: Получить размеры баз данных
  community.postgresql.postgresql_query:
    db: postgres
    query: |
      SELECT datname,
             pg_size_pretty(pg_database_size(datname)) AS size
      FROM pg_database
      WHERE datistemplate = false
      ORDER BY pg_database_size(datname) DESC
  become_user: postgres
  register: db_sizes

- name: Вывести размеры
  ansible.builtin.debug:
    msg: "{{ item.datname }}: {{ item.size }}"
  loop: "{{ db_sizes.query_result }}"
```

1.10.3.5 Автоматизация бэкапов

```yaml
- name: Создать каталог для бэкапов
  ansible.builtin.file:
    path: /var/backups/postgresql
    state: directory
    owner: postgres
    mode: "0700"

- name: Дамп базы данных (ad-hoc)
  community.postgresql.postgresql_db:
    name: myapp_production
    state: dump
    target: "/var/backups/postgresql/myapp_{{ ansible_date_time.date }}.sql.gz"
  become_user: postgres

- name: Настроить ежедневный бэкап через cron
  ansible.builtin.cron:
    name: "Daily PostgreSQL backup"
    minute: "0"
    hour: "2"
    job: >
      pg_dump -Fc myapp_production |
      gzip > /var/backups/postgresql/myapp_$(date +\%Y-\%m-\%d).sql.gz
    user: postgres
```

1.10.3.6 Логическая репликация

```yaml
# На primary: создать публикацию
- name: Создать публикацию
  community.postgresql.postgresql_publication:
    db: myapp_production
    name: myapp_pub
    alltables: true
    state: present
  become_user: postgres
  when: postgresql_role == 'primary'

# На replica: создать подписку
- name: Создать подписку
  community.postgresql.postgresql_subscription:
    db: myapp_analytics
    name: myapp_sub
    publications:
      - myapp_pub
    connparams:
      host: 10.0.1.10
      dbname: myapp_production
      user: replicator
      password: "{{ vault_replicator_pass }}"
  become_user: postgres
  when: postgresql_role == 'analytics'
```

1.10.4 Типичные ошибки и решения

Симптом	Вероятная причина	Диагностика	Решение
`Peer authentication failed for user "ubuntu"`	Нет `become_user: postgres`	Проверить пользователя в задаче	Добавить `become_user: postgres`
`psycopg2 not found`	Не установлен Python-адаптер	`python3 -c "import psycopg2"` на managed node	`apt install python3-psycopg2`
`changed` всегда `true` для `postgresql_query`	`postgresql_query` не идемпотентен	Проверить, есть ли специализированный модуль	Использовать `postgresql_ext`, `postgresql_db` и т.д. Если нет — добавить `changed_when: result.rowcount > 0`
`login_host: localhost` — ошибка аутентификации	`localhost` = TCP, нужен пароль	Проверить pg_hba.conf	Убрать `login_host` для unix socket или добавить пароль
`Module postgresql_set is deprecated`	Используете устаревший модуль	Проверить версию PostgreSQL	Перейти на `postgresql_alter_system` для PG 14+; заменить `name:` на `param:`
`No module named 'psycopg2'`	psycopg2 не установлен на managed node	`python3 -c "import psycopg2"` на целевом хосте	`apt install python3-psycopg2` или `pip install psycopg2`. Psycopg3 тоже поддерживается: `pip install psycopg[binary]`

1.10.5 Что важно запомнить

- Всегда используйте `become_user: postgres` для peer-аутентификации
- `login_host` не указан = unix socket (peer auth). `login_host: localhost` = TCP (нужен пароль)
- `postgresql_alter_system` заменяет `postgresql_set` для PostgreSQL 14+. Параметр называется `param:`, а не `name:`

- `postgresql_query` **не идемпотентен** — используйте специализированные модули. Если без него не обойтись — используйте `changed_when: result.rowcount > 0`
- Пароли — только через Ansible Vault
- Коллекция `community.postgresql` поддерживает как `psycopg2` (>= 2.5.1), так и `psycopg3` (>= 3.1.8). На production используйте `python3-psycopg2` из системных пакетов

1.11 Типичные ошибки и best practices

1.11.1 Почему это важно

Ansible прощает многое на этапе прототипирования, но в production каждая ошибка — это потенциальный downtime. Этот раздел — коллекция паттернов и антипаттернов, собранных из реального опыта.

1.11.2 Практика

1.11.2.1 Ansible Vault: шифрование секретов

Никогда не храните пароли в открытом виде:

```
# Зашифровать строку
ansible-vault encrypt_string 'S3cretP@ss' --name 'pg_password'
# Вывод:
# pg_password: !vault |
#   $ANSIBLE_VAULT;1.1;AES256
#   66386439653966636139613961386662...

# Зашифровать файл
ansible-vault encrypt group_vars/postgresql/vault.yml

# Запуск с расшифровкой
ansible-playbook site.yml --ask-vault-pass

# Или с файлом пароля (для CI/CD)
ansible-playbook site.yml --vault-password-file .vault_pass
```

Структура переменных с Vault:

```
group_vars/postgresql/
├── vars.yml          # Открытые переменные
└── vault.yml         # Зашифрованные пароли
```

```yaml
# group_vars/postgresql/vars.yml
pg_version: "17"
pg_max_connections: 200

# group_vars/postgresql/vault.yml (зашифрован)
vault_app_password: "S3cretP@ss"
vault_replicator_pass: "R3plicat0r!"
vault_monitoring_pass: "M0nitor!"
```

1.11.2.2 Идемпотентность: модули vs raw SQL

```yaml
# ✅ ПРАВИЛЬНО: идемпотентно
- community.postgresql.postgresql_ext:
    name: pg_stat_statements
    db: mydb
    state: present
  become_user: postgres
# Модуль проверяет: расширение уже есть? → ничего не делает

# ❌ НЕПРАВИЛЬНО: changed ВСЕГДА = true
- community.postgresql.postgresql_query:
    query: "CREATE EXTENSION IF NOT EXISTS pg_stat_statements"
    db: mydb
  become_user: postgres
# postgresql_query не знает, изменились ли данные

# ✅ ЕСЛИ postgresql_query необходим — используйте changed_when
- name: Настроить параметр через SQL (нет специального модуля)
  community.postgresql.postgresql_query:
    db: mydb
    query: "INSERT INTO app_config(key, val) VALUES ('timeout', '30') ON
        CONFLICT (key) DO NOTHING"
  register: query_result
  changed_when: query_result.rowcount > 0    # changed только если реально что-
        то изменилось
  become: true
  become_user: postgres
```

1.11.2.3 Проверка перед применением: –check и –diff

```
# Dry-run: показать что изменится, но НЕ применять
ansible-playbook site.yml --check

# Dry-run + показать diff файлов
ansible-playbook site.yml --check --diff

# Запуск только определённых тегов
ansible-playbook site.yml --tags postgresql --limit db-primary

# Пошаговое выполнение (подтверждение каждой задачи)
ansible-playbook site.yml --step
```

1.11.2.4 Тестирование: ansible-lint

```
# Установить
pip install ansible-lint

# Запустить проверку
ansible-lint playbooks/site.yml
# Типичные предупреждения:
# - fqcn[action-core]: Use FQCN for builtin module actions
# - yaml[truthy]: Truthy value should be one of [false, true]
# - no-changed-when: Commands should not change things
```

1.11.3 Типичные ошибки и решения

Симптом	Вероятная причина	Диагностика	Решение
Рестарт PG на всех нодах одновременно	Handler без `serial`	Проверить play	Использовать `serial: 1` для rolling restart
Пароли в Git	Vault не настроен	`git log --diff-filter=A -- '*vault*'`	Зашифровать через `ansible-vault encrypt`, добавить `.vault_pass` в `.gitignore`
Плейбук работает 30+ минут	Не используется pipelining	Проверить `ansible.cfg`	`pipelining = true` в `[ssh_connection]`
Файлы на managed node принадлежат root	Нет `owner: postgres`	`ls -la /etc/postgresql/`	Указать `owner` и `group` в задачах

1.11.4 Что важно запомнить

- **Ansible Vault** для всех паролей — без исключений
- **–check –diff** перед каждым apply в production
- **ansible-lint** — интегрируйте в CI/CD
- **serial: 1** для rolling updates PostgreSQL-кластера
- **become_user: postgres** для всех задач community.postgresql

1.12 Практические рецепты и troubleshooting

1.12.1 Шпаргалка команд

```
# Проверка подключения ко всем хостам
ansible all -m ansible.builtin.ping

# Ad-hoc: версия PostgreSQL на всех серверах
ansible postgresql -m ansible.builtin.command -a "pg_lsclusters --no-header"

# Ad-hoc: проверка доступности PostgreSQL
ansible postgresql -m community.postgresql.postgresql_ping -b --become-user
        postgres

# Запуск плейбука
ansible-playbook -i inventory/hosts.yml site.yml

# Dry-run
ansible-playbook site.yml --check --diff

# Только определённые теги
ansible-playbook site.yml --tags backup

# Только конкретный хост
ansible-playbook site.yml --limit db-primary

# Установка зависимостей
ansible-galaxy install -r requirements.yml

# Проверка синтаксиса
ansible-playbook site.yml --syntax-check

# Линтер
ansible-lint site.yml

# Vault: шифрование
ansible-vault encrypt group_vars/postgresql/vault.yml

# Vault: редактирование зашифрованного файла
ansible-vault edit group_vars/postgresql/vault.yml
```

1.12.2 Карта принятия решений

1.12.2.1 Когда использовать какой инструмент

Критерий	Cloud-init	Ansible	Terraform
Создание ВМ	—	—	✅
Начальная настройка ОС	✅	—	—
Установка PostgreSQL	⚠️ (простые случаи)	✅	—
Управление конфигурацией PG	—	✅	—
Управление пользователями БД	—	✅	—
Повторное применение	❌ (однократно)	✅ (многократно)	✅ (многократно)

1.12.2.2 Выбор роли для PostgreSQL

Критерий	geerlingguy.postgresql	vitabaks.autobase	Собственная роль
Время до production	1 час	2-4 часа	1-2 дня
HA (Patroni)	❌	✅	Зависит от вас
Гибкость	Средняя	Средняя	Максимальная
Поддержка	Community	Community	Ваша команда
Рекомендация	Dev, обучение	Production HA	Нестандартные требования

1.12.2.3 Подключение к PostgreSQL из Ansible

Параметры	Тип соединения	Аутентификация	Когда использовать
`become_user: postgres`, нет `login_host`	Unix socket	peer	Локальное управление (рекомендуется)
`login_host: /var/run/postgresql`	Unix socket	peer	Нестандартный путь к socket
`login_host: localhost`	TCP (127.0.0.1)	password/scram	Удалённое управление через SSH-туннель
`login_host: 10.0.1.10`	TCP	password/scram	Управление удалённым сервером

1.12.3 Диагностика общих проблем

1.12.3.1 Ansible не подключается к хосту

```
# 1. Проверить SSH вручную
ssh deploy@10.0.1.10

# 2. Проверить с verbose-логированием
ansible db-primary -m ping -vvv

# 3. Проверить инвентарь
ansible-inventory --list | python3 -m json.tool

# 4. Проверить ansible.cfg
ansible --version
# Покажет путь к используемому config file
```

1.12.3.2 PostgreSQL-модуль возвращает ошибку

```
# 1. Проверить psycopg2 на managed node
ssh deploy@10.0.1.10 "python3 -c 'import psycopg2;
        print(psycopg2.__version__)'"

# 2. Проверить подключение к PostgreSQL
ssh deploy@10.0.1.10 "sudo -u postgres psql -c 'SELECT version()'"

# 3. Проверить pg_hba.conf
ssh deploy@10.0.1.10 "sudo -u postgres psql -c 'TABLE pg_hba_file_rules'"

# 4. Запустить задачу с отладкой
ansible-playbook site.yml --tags postgresql -vvv
```

1.12.3.3 Плейбук работает медленно

```
# 1. Включить профилирование
# В ansible.cfg:
# [defaults]
# callbacks_enabled = profile_tasks

# 2. Проверить, включён ли pipelining
ansible-config dump | grep PIPELINING

# 3. Увеличить параллельность
ansible-playbook site.yml --forks 20

# 4. Отключить сбор фактов (если не нужны)
# В playbook: gather_facts: false
```

1.12.4 Полный пайплайн: от ВМ до PostgreSQL

```
# Шаг 1: Создать инфраструктуру
terraform apply

# Шаг 2: Установить Ansible-зависимости
ansible-galaxy collection install -r requirements.yml

# Шаг 3: Проверить подключение
ansible all -m ping

# Шаг 4: Dry-run
ansible-playbook -i inventory.yml site.yml --check --diff

# Шаг 5: Применить конфигурацию
ansible-playbook -i inventory.yml site.yml

# Шаг 6: Проверить результат
ansible postgresql -m community.postgresql.postgresql_ping -b --become-user
        postgres
```

1.13 Заключение

1.13.1 Что вы теперь знаете

- ✅ Как использовать **cloud-init** для Day 0 подготовки ВМ под PostgreSQL
- ✅ Как устроена **архитектура Ansible** и почему agentless — это преимущество для DBA
- ✅ Как писать **Ansible-плейбуки** с ролями, handlers и шаблонами Jinja2
- ✅ Как интегрировать **Terraform и Ansible** через динамический инвентарь
- ✅ Как управлять **PostgreSQL через community.postgresql**: базы, пользователи, привилегии, pg_hba.conf, параметры
- ✅ Как шифровать пароли через **Ansible Vault**
- ✅ Как автоматизировать **бэкапы и репликацию** через Ansible
- ✅ Когда использовать **AWX/Tower**, а когда достаточно CLI

1.13.2 Что дальше

На следующих занятиях:

- Patroni для HA-кластеров PostgreSQL
- pgBackRest / WAL-G для production-бэкапов
- Мониторинг PostgreSQL (pg_exporter + Prometheus + Grafana)

Для самостоятельного изучения:

- Запустите примеры из каталога `examples/` презентации (от 01 до 09)
- Напишите свою роль для PostgreSQL, используя модули `community.postgresql`
- Настройте CI/CD пайплайн: `git push` → `ansible-lint` → `ansible-playbook --check` → `ansible-playbook`
- Изучите `molecule` для тестирования ролей в Docker-контейнерах

1.14 Полезные ресурсы

1.14.1 Официальная документация

- Ansible Documentation — полная документация Ansible
- community.postgresql — документация коллекции PostgreSQL
- cloud-init Documentation — документация cloud-init
- terraform-provider-ansible — провайдер Terraform для Ansible

1.14.2 Инструменты

- `geerlingguy.postgresql` — популярная роль для PostgreSQL
- `vitabaks/autobase` — PostgreSQL HA с Patroni через Ansible
- `cloud.terraform` — Ansible-коллекция для интеграции с Terraform
- `ansible-lint` — линтер для Ansible-плейбуков
- `molecule` — тестирование ролей Ansible

1.14.3 Книги и статьи

- Jeff Geerling, «Ansible for DevOps» — лучшая книга для начинающих
- «Ansible: Up and Running» (O'Reilly) — подробное руководство
- PostgreSQL Wiki: Automated testing and deployment — практики автоматизации

Дополнительный материал подготовлен для курса "PostgreSQL Advanced" OTUS.